

TEMELDE JAVASCRIPT PROGRAMI NASIL GÖRÜNÜR

[Kaynak](#)

İÇERİK

1. JavaScript Çalışma Ortamları (Execution Environments) ve Bellek Yönetimi
2. Node.js Ortamında Asenkron STDIN (Standart Girdi) Yönetimi, Tip Dönüşümü ve Kaynak Temizliği
3. JavaScript'te Control Flow (Kontrol Akışı) ve Mantıksal Karşılaştırmalar (Logical Comparisons)
4. JavaScript'te Iterations (Döngüler) ve Sürekli Yürütme (Continuous Execution)

JavaScript Çalışma Ortamları (Execution Environments) ve Bellek Yönetimi

JavaScript (JS), doğası gereği hem istemci (Client-side/Web Browser) hem de sunucu (Server-side/Command Line) tarafında yürütülebilen (Execution) esnek bir dildir. İstemci tarafında testler veya analizler yapmak için web tarayıcılarının Geliştirici Araçları (Web Developer Tools - genelde F12 ile erişilir) içerisindeki konsol kullanılabilir. Ancak siber güvenlikte araç geliştirme, otomasyon betikleri (script) yazma ve arka plan işlemleri (Backend) için komut satırı (CLI) arayüzleri tercih edilir. Bu noktada **Node.js** devreye girer. Bir JavaScript dosyasını terminal üzerinden yürütmek için **node demo.js** komut dizimi kullanılır; buradaki **demo.js** çalıştırılacak kaynak dosyasını temsil eder.

Değişken (Variable) ve Sabit (Constant) Tanımlamaları

Uygulamanın anlık durumunu (State Tracking) sağlamak için bellekte (Memory) verileri saklayacağımız alanlar ayırmamız gerekir. JavaScript'te modern bellek atamaları, verinin yaşam döngüsüne (Life Cycle) bağlı olarak **let** ve **const** anahtar kelimeleri ile yapılır.

Programın akışı içerisinde değeri sürekli güncellenecek olan (Mutable) veriler için **let** kullanılır. Örneğin, bir deneme yanılma oyununda kullanıcının yaptığı hamle sayısı veya o anki tahmini sürekli değişecektir.

```
let tries = 0;  
let guess = 0;
```

Bu kod bloğunda, **tries** (deneme sayısı) değişkenine henüz hiçbir hamle yapılmadığı için başlangıç durumu (Initial State) olarak **0** atanmıştır. Aynı şekilde, **guess** (tahmin) değişkenine de geçerli tahmin aralığının (1-20) dışında olan **0** değeri atanmıştır. **ÖNEMLİ:**

Tahmin değişkenine başlangıçta tesadüfen hedef aralıkta bir değer (Örn: 10) atamak, hedef sayı ile çakışma riski yaratır ve programda mantıksal zafiyetlere (Logical Bug) yol açabilir.

Öte yandan, programın çalışma süresi (Runtime) boyunca değeri asla değişmemesi gereken (Immutable) veriler için **const** (Constant - Sabit) kullanılır. Bellekte salt okunur (Read-Only) bir referans yaratır.

```
const secret = Math.floor(Math.random() * (20)) + 1;
```

Yukarıdaki kod bloğu, sadece bir değişken ataması değil, aynı zamanda hedeflenen aralıkta rastgele (Random) bir tam sayı üretilmesinin standart bir matematiksel algoritmasıdır. Arka plandaki çalışma mantığı (Execution Flow) şu adımlarla gerçekleşir:

1. Sistem ilk olarak **Math.random()** metodunu çağırır. Bu fonksiyon, **0 (dahil)** ile **1 (hariç)** arasında ondalıklı (Floating Point) bir sayı üretir (Örn: 0.372).
2. Üretilen bu ham ondalıklı değer **20** ile çarpılarak hedef aralık genişletilir (**0** ile **19.999...** arasına çekilir). Çıktı örneğin 7.44 olur.
3. Ancak bize bir tam sayı (Integer) gereklidir. Devreye giren **Math.floor()** metodu, bu değeri her zaman bir alt tam sayıya yuvarlayarak ondalık kısmı budar (Truncation). Böylece elimizde **0 ile 19** arasında kusursuz bir tam sayı kalır.
4. Son olarak, elde edilen değere **+ 1** eklenerek (Offset Shift), aralık istenilen **1 ile 20** (1 ve 20 dahil) bandına oturtulur ve bu değer **secret** sabitine yazılır.

Standart Çıktı (STDOUT) İşlemleri

Arka planda bellek atamaları ve algoritmalar çalıştıktan sonra, kullanıcının veya sistemi izleyen analistin bilgilendirilmesi gerekir. JavaScript'te Standart Çıktı (STDOUT) akışına veri basmak için **console.log()** metodu kullanılır.

```
console.log("I'm thinking of a number between 1 and 20");
```

Bu komut yürütüldüğünde, çift tırnak (String) içerisine alınmış metin dizisi doğrudan terminal veya tarayıcı konsoluna yazdırılarak programın hazır olduğu bilgisi kullanıcıya aktarılmış olur. Çıktı işlemleri, debug (hata ayıklama) süreçlerinde de arka planda dönen **let** ve **const** değerlerini anlık olarak gözlemlemek için kritik bir role sahiptir.

Node.js Ortamında Asenkron STDIN (Standart Girdi) Yönetimi, Tip Dönüşümü ve Kaynak Temizliği

Siber güvenlikte ve modern geliştirme ortamlarında bir komut satırı aracı (CLI tool) inşa ederken, kullanıcıdan veya sistemden alınan verilerin (girdilerin) doğru ve güvenli bir şekilde işlenmesi hayati önem taşır. JavaScript'in komut satırı çalışma zamanı olan **Node.js**, doğası gereği web sunucuları ve asenkron (eşzamanlı olmayan) işlemler için tasarlanmıştır. Python'daki `input()` fonksiyonunun aksine, Node.js varsayılan olarak kullanıcının terminale

bir girdi girmesini bekleyerek (blocking) programın akışını durdurmaz. Bu asenkron doğayı manipüle etmek ve programı bir kullanıcı girdisi alana kadar beklemeye zorlamak için özel kütüphanelere ve arayüzlere (Interface) ihtiyacımız vardır.

I/O (Girdi/Çıktı) Arayüzünün Kurulması ve Asenkron Bekletme

Node.js ortamında terminal ile iletişim kurabilmek adına standart girdi (**stdin**) ve standart çıktı (**stdout**) akışlarını yöneten modülleri programımıza dahil etmeliyiz.

```
import * as readline from "node:readline/promises";
import { stdin as input, stdout as output } from "node:process";
const rl = readline.createInterface({ input, output });
```

Bu kod bloğunun arka plandaki teknik derinliği şu şekildedir: İlk satırda, programın terminal üzerinden soru sorup cevap almasını sağlayan **readline** modülü dahil edilmektedir. Burada özellikle **/promises** uzantısının kullanılması çok kritiktir. Bu sayede I/O işlemleri "Promise" tabanlı hale gelir ve asenkron akışı duraklatabilen **await** anahtar kelimesi ile uyumlu çalışır. İkinci satırda, doğrudan çekirdek sistem süreçlerinden (Node Process) **stdin** (klavye/girdi) ve **stdout** (ekran/çıktı) modülleri içeri aktarılır ve kodun okunabilirliğini artırmak adına sırasıyla **input** ve **output** olarak yeniden adlandırılır. Üçüncü satırda ise **readline.createInterface()** metodu kullanılarak, işletim sisteminin girdi/çıktı kanalları ile programımız arasında sürekli açık kalan bir iletişim köprüsü (**rl** nesnesi) kurulur.

Kullanıcı Girdisinin Yakalanması ve Güvenli Tip Dönüşümü (Type Casting)

İletişim kanalını açtıktan sonra, programın çalışma zamanını (Execution) duraklatıp kullanıcının veri girmesini sağlamalıyız.

```
const text = await rl.question("Take a guess: ");
guess = parseInt(text, 10);
```

ÖNEMLİ: Burada kullanılan **await** komutu sıradan bir bekleme işlemi değildir. Node.js'in olay döngüsünü (Event Loop) engellemeden, sadece bu spesifik kod bloğunun çalışmasını Promise çözümlene (kullanıcı "Enter" tuşuna basana) kadar askıya alır (Suspend). Kullanıcının terminale girdiği veri, her zaman karakter dizisi (String) formatında döner ve **text** sabitine (Constant) atanır.

İkinci satırdaki **parseInt(text, 10)** fonksiyonu ise siber güvenlik ve yazılım kararlılığı açısından kritik bir **Type Casting (Tip Dönüşümü)** işlemidir. Sistemden alınan String veriyi, matematiksel karşılaştırmalarda kullanabilmek için Integer'a (Tam Sayı) dönüştürürüz. Buradaki **10** parametresi (Radix/Base) onluk sayı tabanını ifade eder. **Neden bu parametreyi kullanıyoruz?** Eğer kullanıcı girdisi "0" ile başlarsa (Örn: "015"), bazı eski

JavaScript motorları bunu sekizlik (Octal) tabanda yorumlamaya çalışabilir ve programın yanlış matematiksel hesaplamalar yapmasına (Logic Flaw) neden olabilir. Tabanı açıkça 10 olarak belirtmek, belirsizlikleri ortadan kaldırır ve girdi manipülasyonlarını engeller.

Kaynak Yönetimi ve Güvenli Temizlik (Clean Execution)

Sistem seviyesinde bir I/O arayüzü açmak (bizim durumumuzda **rl** nesnesi), işletim sistemi kaynaklarını (Memory & File Descriptors) tüketir. İşlem bittiğinde bu kaynakların serbest bırakılmaması, bellek sızıntılarına (Memory Leak) veya programın asılı kalarak (Hanging Process) kapanmamasına yol açar.

```
try {
  // ... Girdi alma ve işleme kodları ...
  const text = await rl.question("Take a guess: ");
  // ...
} finally {
  rl.close();
}
```

Bu noktada **try...finally** bloğu devreye girer. **try** bloğu içindeki kodlar çalıştırılırken beklenmedik bir hata (Exception) meydana gelse veya süreç çökse bile, dilin mimarisi gereği **finally** bloğu her koşulda mutlak surette çalıştırılır. **rl.close()** komutunun **finally** içine yerleştirilmesi, program hangi koşulda sonlanırsa sonlansın açık kalan iletişim kanalının kapatılmasını ve sistem kaynaklarının güvenli bir şekilde iade edilmesini garanti altına alır.

İlk Taslağın Birleştirilmesi: guess_v1.js

Tüm bu I/O yönetimi, değişken tanımlamaları, rastgele sayı üretimi ve kaynak temizliği adımlarını bir araya getirdiğimizde, `/home/ubuntu/JavaScript-Demo` dizininde bulunan `guess_v1.js` betiği (script) ortaya çıkar.

```
import * as readline from "node:readline/promises";
import { stdin as input, stdout as output } from "node:process";

const rl = readline.createInterface({ input, output });

try {
  const secret = Math.floor(Math.random() * (20)) + 1; // 1 <= secret <=
20 aralığında hedef sabiti
  let tries = 0;
  let guess = 0; // Hedef olamayacak başlangıç durumu (Initial State)

  console.log("I'm thinking of a number between 1 and 20");

  const text = await rl.question("Take a guess: "); // I/O'yu beklet
  (await) ve girdiyi String olarak al
  guess = parseInt(text, 10); // Girdiyi Base-10 Integer'a güvenli şekilde
```

dönüştür

```
    tries = tries + 1; // Durum takibi: Deneme sayacını artır
  } finally {
    rl.close(); // İşlem bitiminde (veya hatada) kaynakları mutlak surette
    serbest bırak
  }
```

DİKKAT: Mevcut kod bloğu, bellek yönetimi, asenkron I/O işlemleri ve değişken atamaları açısından kusursuz bir mimariye sahiptir. Ancak betiğin en büyük mantıksal eksikliği, Control Flow (Kontrol Akışı) eksikliğidir. Kullanıcı verisi belleğe alınmakta, dönüştürülmekte ancak arka planda üretilen **secret** ile karşılaştırılıp kullanıcıya herhangi bir geri bildirim (Feedback/Output) sunulmamaktadır. Kod, işlemi sessizce bitirip kapanmaktadır. Bir sonraki aşamada bu verilerin işlenip karar ağaçlarına (Conditional Statements) dökülmesi gerekmektedir.

JavaScript'te Control Flow (Kontrol Akışı) ve Mantıksal Karşılaştırmalar (Logical Comparisons)

Önceki aşamada I/O arayüzünü kurarak kullanıcıdan veriyi bellek alanına aktarmayı ve güvenli bir Type Casting (Tip Dönüşümü) yapmayı başardık. Ancak mevcut durumda script, alınan veriyi değerlendirme veya bu veriye göre bir eylem planı oluşturma yeteneğinden yoksundur. Bir siber güvenlik aracının veya otomasyon betiğinin "karar verebilmesi", işlemcide **Control Flow (Kontrol Akışı)** yaratılmasına bağlıdır. JavaScript'te bu akış, birbirini dışlayan (Mutually Exclusive) senaryoları sırayla test eden **Conditional Statements (Koşullu İfadeler)** ile sağlanır.

Kullanıcının tahmini (`guess`) ile sistemde üretilen hedef sabitini (`secret`) karşılaştırmak için tasarladığımız karar ağacını (Decision Tree) ve JavaScript sentaksını inceleyelim.

```
// if / else if / else yapılarıyla Mantıksal Karşılaştırma Bloğu
if (guess < 1 || guess > 20) {
    console.log("That number is out of range. Try again.");
} else if (guess < secret) {
    console.log("Too low, try again.");
} else if (guess > secret) {
    console.log("Too high, try again.");
} else {
    console.log("You got it in", tries, "tries!");
}
```

Yukarıdaki kod bloğunun işlemci (CPU) ve yorumlayıcı (Interpreter) tarafındaki teknik işleyişi şöyledir:

Karar mekanizması her zaman yukarıdan aşağıya (Top-to-Bottom) çalışır. Eğer zincirdeki bir koşul `true` (doğru) olarak değerlendirilirse, JavaScript motoru o bloğun içindeki komutu

çalıştırır (Execute) ve zincirin geri kalanını **okumadan atlar**. Bu, kaynak tüketimini engelleyen ve uygulamanın beklenmeyen durumlara girmesini önleyen kritik bir optimizasyondur.

1. `if (guess < 1 || guess > 20)` (**Sınır Kontrolü - Boundary Check**): Scriptin kontrol ettiği ilk savunma hattıdır. Burada `||` (**Logical OR - Mantıksal VEYA**) operatörü kullanılmıştır. Kullanıcının girdisi 1'den küçükse **VEYA** 20'den büyükse, koşul `true` döner. Program "Out of range" (Kapsam dışı) uyarısını basar. Sınır dışı (Out of Bounds) kontrollerinin her zaman en başta yapılması, arka plandaki ana mantığın geçersiz verilerle yorulmasını engeller.
2. `else if (guess < secret)` (**Alt Sınır Kontrolü**): `else if` yapısı, yalnızca bir üstündeki `if` koşulu `false` döndüğünde, yani veri geçerli aralıkta (1-20) olduğunda devreye girer. Tahminin, hedeften küçük olup olmadığını sorgular.
3. `else if (guess > secret)` (**Üst Sınır Kontrolü**): Eğer tahmin geçerli aralıktaysa ancak hedeften küçük değilse, bu kez hedeften büyük olup olmadığı kontrol edilir.
4. `else` (**Varsayılan/Düşüş Durumu - Fallback**): Eğer tüm üst koşullar `false` döndüyse, bu bloğa düşülür. Matematiksel olarak; eğer sayı sınırların dışına çıkmamışsa, hedeften küçük değilse ve hedeften büyük de değilse, geriye kalan tek olasılık hedefe **eşit** olmasıdır. Bu yüzden tekrar bir eşittir (`==` veya `===`) kontrolü yazmak gereksizdir. Program başarı mesajını ekrana basar.

İkinci Taslağın Tamamlanması: guess_v2.js

Bu mantıksal kontrol mekanizmasını daha önce yazdığımız I/O (Girdi/Çıktı) ve bellek yönetimi kodlarına entegre ettiğimizde, hedef sistemdeki `/home/ubuntu/JavaScript-Demo` dizininde bulunan `guess_v2.js` dosyası elde edilir.

```
import * as readline from "node:readline/promises";
import { stdin as input, stdout as output } from "node:process";

const rl = readline.createInterface({ input, output });

try {
  const secret = Math.floor(Math.random() * (20)) + 1; // 1 <= secret <= 20

  let tries = 0;
  let guess = 0; // Hedef olamayacak başlangıç durumu

  console.log("I'm thinking of a number between 1 and 20");

  const text = await rl.question("Take a guess: "); // I/O'yu beklet ve STDIN'den veri al
  guess = parseInt(text, 10); // Girdiyi Base-10 Integer'a dönüştür

  tries = tries + 1; // Durum takibi: Deneme sayacını 1 artır
```

```
// Mantıksal Karşılaştırma Ağacı (Decision Tree)
if (guess < 1 || guess > 20) {
    console.log("That number is out of range. Try again.");
} else if (guess < secret) {
    console.log("Too low, try again.");
} else if (guess > secret) {
    console.log("Too high, try again.");
} else {
    console.log("You got it in", tries, "tries!");
}
} finally {
    rl.close(); // Süreç sonunda I/O kaynaklarını (File Descriptors/Memory)
serbest bırak
}
```

Bu kod bloğu, veriyi alır, doğrular, karşılaştırır ve çıktıyı üretir. Artık tamamen işlevsel bir scriptimiz var. Ancak, scriptin çalışma anındaki (Runtime) davranışını gözlemlediğimizde kritik bir eksiklik (Limitation) ortaya çıkar.

Terminal Execution (Çalışma Zamanı Analizi) ve Mevcut Sınırlamalar

Yazdığımız betiği Node.js üzerinden çağırdığımızda sistemin verdiği tepkiyi inceleyelim:

```
ubuntu@tryhackme:~/JavaScript-Demo$ node guess_v2.js
I'm thinking of a number between 1 and 20
Take a guess: 10
Too low, try again.
ubuntu@tryhackme:~/JavaScript-Demo$
```

Terminal çıktısını satır satır yorumladığımızda; Sistem `node guess_v2.js` komutu ile süreci (Process) başlattı. I/O arayüzü beklemeye geçti ve kullanıcıdan 10 girdisi alındı. Script karar ağacını işledi ve 10 sayısının `secret` değerinden küçük olduğunu tespit edip "Too low, try again." çıktısını STDOUT'a bastı.

ÖNEMLİ: Çıktının hemen ardından işletim sisteminin terminal promptu (`ubuntu@tryhackme...$`) geri döndü. Bunun teknik anlamı şudur: Script görevini tamamladı, `finally` bloğu çalışıp `rl.close()` ile I/O kanalını kapattı ve **süreç (Process)** **sonlandırıldı**. Program tek seferlik çalışma (Single Execution) mantığıyla dizayn edildiği için kullanıcıya vadettiği gibi "Try again" (Tekrar dene) şansını aslında sunmamaktadır. Etkileşimli bir araç (Interactive Tool) inşa etmek ve kullanıcı başarıya ulaşana kadar süreci hayatta tutmak için, kontrol akışına Iterations (Döngüler - Loops) mantığının entegre edilmesi zorunludur.

JavaScript'te Iterations (Döngüler) ve Sürekli Yürütme (Continuous Execution)

Bir önceki evrede geliştirdiğimiz kontrol akışı (Control Flow) mantıksal olarak doğru çalışsa da, siber güvenlik araçlarında ve interaktif CLI (Command Line Interface) uygulamalarında en büyük gereksinim olan "süreklilik" (Continuity) eksikti. Script tek bir işlem yapıyor ve işletim sistemi sürecini (Process) sonlandırıyordu. Uygulamayı, kullanıcı doğru veriyi girene kadar hayatta tutmak ve aynı mantıksal işlemleri tekrarlatmak için **Iterations (Döngüler)** mimarisine geçiş yapmamız gerekir.

Döngü Mantığı ve Sıkı Eşitsizlik (Strict Inequality)

JavaScript'te bir kod bloğunu belirli bir koşul geçerliliğini koruduğu sürece tekrar tekrar yürütmek için **while loop (while döngüsü)** kullanılır. Bizim senaryomuzda, uygulamanın çalışmaya devam etmesi için gereken koşul; kullanıcının tahmininin (`guess`), sistemin belirlediği hedef sayıya (`secret`) eşit **olmamasıdır**.

```
// Kullanıcı doğru sayıyı bulana kadar süreci hayatta tutan Ana Döngü
while (guess !== secret) {
  // Döngü Gövdesi (Loop Body): Tekrarlanacak yönergeler
}
```

ÖNEMLİ: Burada kullanılan `!==` (**Strict Not Equal - Sıkı Eşit Değildir**) operatörü, JavaScript'e özgü son derece kritik bir güvenlik ve kararlılık detaydır. Sadece değerlerin (value) değil, veri tiplerinin (type) de eşitsizliğini kontrol eder. Eğer kullanıcı girdisini daha önce `parseInt()` ile Integer'a (Tam Sayı) dönüştürmemiş olsaydık, String tipindeki `"10"` ile Integer tipindeki `10`, gevşek eşitsizlik (`!=`) operatöründe hatalara yol açabilirdi. Sıkı karşılaştırma kullanmak, her zaman Type Safety (Tip Güvenliği) sağlar ve mantıksal zafiyetleri (Logic Flaws) önler.

Ana Döngünün (Loop Body) İnşası ve Durum Güncellemesi

Döngü gövdesi içerisine, programın her bir iterasyonda (tekrarda) yapması gereken tüm işlemler sırasıyla yerleştirilmelidir. Döngü her çalıştığında; kullanıcıdan yeni veri alınmalı, tipe dönüştürülmeli, durum sayacı (State Tracker - `tries`) güncellenmeli ve karar ağacından geçirilerek çıktı üretilmelidir.

```
// guess değeri secret değerine sıkı sıkıya eşit olmadığı SÜRECE bu bloğu çalıştır
while (guess !== secret) {
  const text = await rl.question("Take a guess: "); // Asenkron bekleme:
  // Olay Döngüsünü (Event Loop) dondurmadan girdiyi bekle
  guess = parseInt(text, 10); // Güvenli Tip Dönüşümü (String -> Base 10 Integer)

  tries = tries + 1; // Durum Takibi: Kullanıcı hamle yaptı, sayacı 1 artır

  // Control Flow: Sınır Kontrolleri ve Karşılaştırma Ağacı
```



```
if (guess < 1 || guess > 20) {
    console.log("That number is out of range. Try again.");
} else if (guess < secret) {
    console.log("Too low, try again.");
} else if (guess > secret) {
    console.log("Too high, try again.");
} else {
    console.log("You got it in", tries, "tries!");
}
}
```

Bu döngünün teknik kalbi, `await rl.question()` metodunun döngü içerisinde yer almasıdır. JavaScript'in asenkron yapısı sayesinde, döngü her başa sardığında program `await` satırında **askıya alınır (Suspended)**. Süreç kapanmaz, CPU döngüleri (CPU Cycles) boşa harcanmaz, sistem sadece yeni bir I/O (Girdi/Çıktı) tetiklenmesini (Kullanıcının Enter'a basmasını) bekler.

Üçüncü Taslağın Tamamlanması: guess_v3.js

Tüm modül içe aktarımlarını (Imports), değişken atamalarını, I/O kurulumunu, kaynak yönetimini ve ana döngüyü (Main Loop) birleştirdiğimizde, uygulamanın hedef sistemde `/home/ubuntu/JavaScript-Demo` dizini altında bulunan nihai hali `guess_v3.js` ortaya çıkar.

```
import * as readline from "node:readline/promises";
import { stdin as input, stdout as output } from "node:process";

const rl = readline.createInterface({ input, output });

try {
    const secret = Math.floor(Math.random() * (20)) + 1; // 1 <= secret <=
    20 aralığında hedef sabiti
    let tries = 0;
    let guess = 0; // Hedef olamayacak başlangıç durumu, böylece döngü ilk
    kontrolde direkt içine girer

    console.log("I'm thinking of a number between 1 and 20");

    // Ana Döngü (Main Loop)
    while (guess !== secret) {
        const text = await rl.question("Take a guess: "); // I/O'yu beklet
        ve STDIN'den veri al
        guess = parseInt(text, 10); // Girdiyi Base-10 Integer'a dönüştür

        tries = tries + 1; // Sayaç güncellemesi

        // Mantıksal Karşılaştırma Ağacı (Decision Tree)
        if (guess < 1 || guess > 20) {
            console.log("That number is out of range. Try again.");
        }
    }
}
```

```
    } else if (guess < secret) {
        console.log("Too low, try again.");
    } else if (guess > secret) {
        console.log("Too high, try again.");
    } else {
        console.log("You got it in", tries, "tries!");
    }
}
} finally {
    rl.close(); // Döngü kırıldığında (başarı durumunda) veya kritik hatada
I/O kanalını kapat ve süreci sonlandır
}
```

Terminal Execution (Çalışma Zamanı Analizi) ve Çıktı Yorumlama

Bu kodun çalışma zamanındaki (Runtime) davranışını ve arka planda dönen mantığı anlamak için terminal (CLI) loglarını satır satır analiz edelim:

```
ubuntu@tryhackme:~/JavaScript-Demo$ node guess_v3.js
I'm thinking of a number between 1 and 20
Take a guess: 10
Too low, try again.
Take a guess: 15
Too high, try again.
Take a guess: 13
Too low, try again.
Take a guess: 14
You got it in 4 tries!
```

Satır Satır Çıktı (Log) Analizi:

- `node guess_v3.js` : İşletim sisteminde Node.js süreci başlatıldı. Hafızada `secret` atandı (Bu senaryoda arka planda 14 üretildiğini çıktıdan anlıyoruz). `guess = 0` olduğu için `0 !== 14` koşulu **true** döndü ve döngü bloğuna girildi.
- `Take a guess: 10` : İlk I/O beklemesi. Kullanıcı 10 girdi. `guess` 10 oldu, `tries` 1 oldu. Karar ağacında `10 < 14` (`guess < secret`) tetiklendi.
- `Too low, try again.` : Çıktı basıldı. Döngü sonuna ulaşıldı ve başa sardı. `10 !== 14` hala **true**. Süreç hayatta.
- `Take a guess: 15` : İkinci I/O beklemesi. Kullanıcı 15 girdi. `tries` 2 oldu. `15 > 14` (`guess > secret`) tetiklendi.
- `Too high, try again.` : Çıktı basıldı. Döngü başa sardı. `15 !== 14` hala **true**.
- `Take a guess: 13` : Üçüncü I/O. `tries` 3 oldu. `13 < 14` tetiklendi.
- `Too low, try again.` : Çıktı basıldı. Döngü başa sardı. `13 !== 14` hala **true**.

8. `Take a guess: 14` : Dördüncü ve son I/O. Kullanıcı 14 girdi. `guess 14` oldu. `tries 4` oldu. Sınır kontrolleri ve küçüktür/büyüktür kontrolleri `false` döndüğü için kontrol akışı doğrudan `else` bloğuna düştü.
9. `You got it in 4 tries!` : Başarı mesajı STDOUT'a basıldı.
10. **Döngü Kırılımı (Loop Break) ve Kaynak Temizliği:** Kod bloğu sonuna gelindiğinde döngü tekrar başa sarmak için koşulu kontrol etti: `14 != 14` . Bu ifade matematiksel ve mantıksal olarak artık **false** değerini ürettiği için döngü bloğu atlandı (Bypassed). Akış doğrudan `finally` bloğuna geçti, `rl.close()` çalıştırılarak bellek/file descriptor kaynakları sisteme iade edildi ve süreç temiz bir şekilde (Clean Exit) sonlandı. (Terminal promptunun geri döndüğü an burasıdır).

DİKKAT: Hedef sistemde yer alan `guess_v4.js` betiği bu uygulamanın daha da geliştirilmiş versiyonunu barındırır. İleri düzey otomasyon ve güvenlik araçları geliştirirken kod yapısının daha modüler (fonksiyonel) hale getirilmesi için bu tür evrimleri incelemek sektör standartlarını kavramak adına kritik bir adımdır.